

Python CSV

A CSV stands for "comma separated values", which is defined as a simple file format that uses specific structuring to arrange tabular data.

It stores tabular data such as spreadsheets or databases in plain text and has a common format for data interchange. A CSV file opens into the Excel sheet, and the rows and columns data define the standard format.

Writing Data in a CSV File

We can use the `CSV.writer()` function to write data to a CSV file. It returns a writer object we can use to write rows to the CSV file. We can write rows by calling the `writer ()` method on the writer object.

Example:

```
import csv
```

```
data = [['Name', 'Age', 'Country'],  
        ['ABC', '25', 'USA'],
```

```
['XYZ', '30', 'Canada'],  
['RST', '35', 'Australia']  
]
```

with open('data.csv', 'w') as file:

```
writer = csv.writer(file)
```

```
for row in data:
```

```
writer.writerow(row)
```

In this program, we create a list of lists called data, where each inner list represents a row of data.

We then open the file data.csv in write mode and create a CSV.writer object using the CSV.writer function.

We then iterate over the rows in data using a for loop and write each row to the CSV file using the writer method.

Reading Data from a CSV File

We can use the `CSV.reader` function to read a CSV file. This function returns a reader object that we can use to repeat over the rows in the CSV file. Each row is returned as a list of values, where each value corresponds to a column in the CSV file.

Example

```
import csv

with open('data.csv', 'r') as file:

    reader = csv.reader(file)

    for row in reader:

        print(row)
```

Here, we open the file `data.csv` in read mode and create a `csv.reader` object using the `csv.reader()` function. We then iterate over the rows in the CSV file using a for loop and print each row to the console.

Exception Handling

Python Exceptions

An exception can be defined as an unusual condition in a program resulting in an interruption in the flow of the program.

Whenever an exception occurs, the program stops the execution, and thus the other code is not executed. Therefore, an exception is the run-time errors that are unable to handle to Python script. An exception is a Python object that represents an error.

Python Exceptions are an important aspect of error handling in Python programming. When a program encounters an unexpected situation or error, it may raise an exception, which can interrupt the normal flow of the program.

To handle exceptions in Python, we use the **try** and **except** statements. The **try** statement is used to enclose the code that may raise an exception, while the **except** statement is used to define a block of code that should be executed when an exception occurs.

For example, consider the following code:

try:

```
x = int(input("Enter a number: "))
```

```
y = 10 / x
```

```
print("Result:", y)
```

except ZeroDivisionError:

```
print("Error: Division by zero")
```

except ValueError:

```
print("Error: Invalid input")
```

Different types of exceptions in python:

In Python, there are several built-in Python exceptions that can be raised when an error occurs during the execution of a program. Here are some of the most common types of exceptions in Python:

SyntaxError: This exception is raised when the interpreter encounters a syntax error in the code, such as a misspelled keyword, a missing colon, or an unbalanced parenthesis.

TypeError: This exception is raised when an operation or function is applied to an object of the wrong type, such as adding a string to an integer.

NameError: This exception is raised when a variable or function name is not found in the current scope.

IndexError: This exception is raised when an index is out of range for a list, tuple, or other sequence types.

KeyError: This exception is raised when a key is not found in a dictionary.

ValueError: This exception is raised when a function or method is called with an invalid argument or input, such as trying to convert a string to an integer when the string does not represent a valid integer.

AttributeError: This exception is raised when an attribute or method is not found on an object, such as trying to access a non-existent attribute of a class instance.

IOError: This exception is raised when an I/O operation, such as reading or writing a file, fails due to an input/output error.

ZeroDivisionError: This exception is raised when an attempt is made to divide a number by zero.

ImportError: This exception is raised when an import statement fails to find or load a module.

SQLite Python

Python provides two popular interfaces for working with the SQLite database library: PySQLite and APSW. Each interface targets a set of different needs.

PySQLite

The PySQLite provides a standardized Python DBI API 2.0 compliant interface to the SQLite database. If your application needs to support not only the SQLite database but also other databases such as MySQL, PostgreSQL, and Oracle, the PySQLite is a good choice.

PySQLite is a part of the Python standard library since Python version 2.5

APSW

If your application needs to support only the SQLite database, you should use the APSW module, which is known as Another Python SQLite Wrapper.

The APSW provides the thinnest layer over the SQLite database library. The APSW is designed to mimic the native SQLite C, therefore, whatever you can do in SQLite C API, you can do it also from Python.

Besides covering the SQLite library, the APSW provides many low-level features including the ability to create user-defined aggregate, function, and collations from Python. It even allows you to write a virtual table implementation using Python.

We will use the PySQLite to work with the SQLite database library using Python.

- **Creating an SQLite database** from a Python program: shows you how to create a new SQLite database from a Python program using the sqlite3 module.
- **Create tables in SQLite database** using Python: shows you step by step how to create tables in an SQLite database from a Python program.
- **Inserting data into the SQLite database** in Python: walks you through the steps of inserting data into a table in SQLite database using Python.
- **Updating data in the SQLite database** using Python: learns how to update existing data in the SQLite database using Python.
- **Selecting data:** this tutorial shows you how to query data in an SQLite database from a Python program.
- **Deleting data** from a Python program: guides you on how to delete one or more rows in a table from a table using Python sqlite3 module API.

Download and install db browser for sqlite

Open below link

<https://sqlitebrowser.org/dl/>

Select your windows version

Windows

Our latest release (3.13.1) for Windows:

- [DB Browser for SQLite - Standard installer for 32-bit Windows](#)
- [DB Browser for SQLite - .zip \(no installer\) for 32-bit Windows](#)
- [DB Browser for SQLite - Standard installer for 64-bit Windows](#)
- [DB Browser for SQLite - .zip \(no installer\) for 64-bit Windows](#)

Download and install

```
sqlite3.connect(database [,timeout ,other optional arguments])
```

This API opens a connection to the SQLite database file. If database is opened successfully, it returns a connection object.

1. timeout parameter

*When a database is accessed by multiple connections, and one of the processes modifies the database, the SQLite database is locked until that transaction is committed. The timeout parameter specifies how long the connection should wait for the lock to go away until raising an exception. The default for the timeout parameter is **5.0 (five seconds)**.*

If the given database name does not exist then this call will create the database. You can specify filename with the required path as well if you want to create a database anywhere else except in the current directory.

2 Connection.cursor([cursorClass])

This routine creates a cursor which will be used throughout of your database programming with Python. This method accepts a single optional parameter cursorClass. If supplied, this must be a custom cursor class that extends sqlite3.Cursor.

3 cursor.execute(sql [, optional parameters])

This routine executes an SQL statement. The SQL statement may be parameterized (i. e. placeholders instead of SQL literals). The sqlite3 module supports two kinds of placeholders: question marks and named placeholders (named style).

For example – `cursor.execute("insert into people values (?, ?)", (who, age))`

4 connection.execute(sql [, optional parameters])

This routine is a shortcut of the above execute method provided by the cursor object and it creates an intermediate cursor object by calling the cursor method, then calls the cursor's execute method with the parameters given.

5 cursor.executemany(sql, seq_of_parameters)

This routine executes an SQL command against all parameter sequences or mappings found in the sequence sql.

6 connection.executemany(sql[, parameters])

This routine is a shortcut that creates an intermediate cursor object by calling the cursor method, then calls the cursor's executemany method with the parameters given.

7 cursor.executescript(sql_script)

This routine executes multiple SQL statements at once provided in the form of script. It issues a COMMIT statement first, then executes the SQL script it gets as a parameter. All the SQL statements should be separated by a semi colon (;).

8 connection.executescript(sql_script)

This routine is a shortcut that creates an intermediate cursor object by calling the cursor method, then calls the cursor's executescript method with the parameters given.

9 connection.total_changes()

This routine returns the total number of database rows that have been modified, inserted, or deleted since the database connection was opened.

10 connection.commit()

This method commits the current transaction. If you don't call this method, anything you did since the last call to `commit()` is not visible from other database connections.

11 `connection.rollback()`

This method rolls back any changes to the database since the last call to `commit()`.

12 `connection.close()`

This method closes the database connection. Note that this does not automatically call `commit()`. If you just close your database connection without calling `commit()` first, your changes will be lost!

13 `cursor.fetchone()`

This method fetches the next row of a query result set, returning a single sequence, or `None` when no more data is available.

14 `cursor.fetchmany([size = cursor.arraysize])`

This routine fetches the next set of rows of a query result, returning a list. An empty list is returned when no more rows are available. The method tries to fetch as many rows as indicated by the size parameter.

15 **cursor.fetchall()**

This routine fetches all (remaining) rows of a query result, returning a list. An empty list is returned when no rows are available.

- **Create Database**

```
import sqlite3
```

```
conn = sqlite3.connect("sqlite.db")
```

- **Create Table**

```
import sqlite3

conn = sqlite3.connect("sqlite.db")

conn.execute("""

Create table student(

st_id INTEGER PRIMARY KEY AUTOINCREMENT,

st_nm VARCHAR(50),

st_class VARCHAR(20),

st_email VARCHAR(80)

)

""")

conn.close()
```

- **Insert Records**

```
import sqlite3

conn = sqlite3.connect("sqlite.db")

ins = ""

INSERT INTO student(st_nm, st_class, st_email)

VALUES("abc", "12", "abc@gmail.com")

""
```

```
conn.execute(ins)
```

```
conn.commit()
```

```
conn.close()
```

- **Select Records**

```
import sqlite3
```

```
conn = sqlite3.connect("sqlite.db")
```

```
data= conn.execute("select * from student order by st_nm DESC limit 2")
```

```
for m in data:
```

```
    print(m)
```

```
conn.close()
```

- **Delete Record**

```
import sqlite3

conn = sqlite3.connect("sqlite.db")

data= conn.execute("select * from student order by st_nm DESC")

for m in data:

    print(m)

id = input("id to delete:")

conn.execute("delete from student where id=" + id)

conn.commit()

data1 = conn.execute("select * from student order by st_nm DESC")

for m in data1:

    print(m)

conn.close()
```

- **Update Records**

```
import sqlite3

conn = sqlite3.connect("sqlite.db")

data = conn.execute("select * from student order by st_nm DESC")

for m in data:

    print(m)

conn.execute("update student set st_nm = 'xyz2' where st_id=5")

conn.commit()

data1 = conn.execute("select * from student order by st_nm DESC")

for m in data1:

    print(m)

conn.close()
```

- **Filter/Search Record**

```
import sqlite3

conn = sqlite3.connect("sqlite.db")

nm = input("student name:")
data = conn.execute("select * from student where st_nm = '"+nm+"'")

for m in data:
    print(m)

conn.close()
```

- **Search Distinct**

```
import sqlite3  
conn = sqlite3.connect("sqlite.db")  
data= conn.execute("select distinct st_class from student")  
  
for m in data:  
    print(m)  
conn.close()
```